Georgia Tech School of Electrical and Computer Engineering
College of Engineering

SYNERGY

http://synergy.ece.gatech.edu

GAMMA: Mapping Space Exploration via Optimization

Sheng-Chun (Felix) Kao

Georgia Institute of Technology

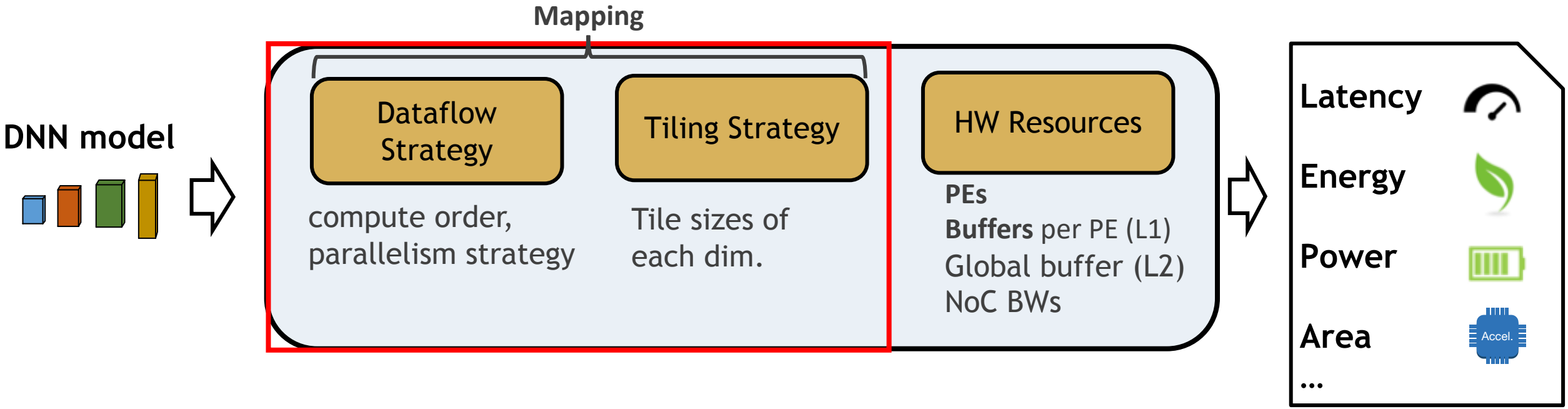
MICRO 2020

Date: Oct. 17, 2020

S.-C. Kao, T Krishna, "GAMMA: Automating the HW Mapping of DNN Models on Accelerators via Genetic Algorithm", ICCAD, 2020

Code available: https://github.com/maestro-project/gamma

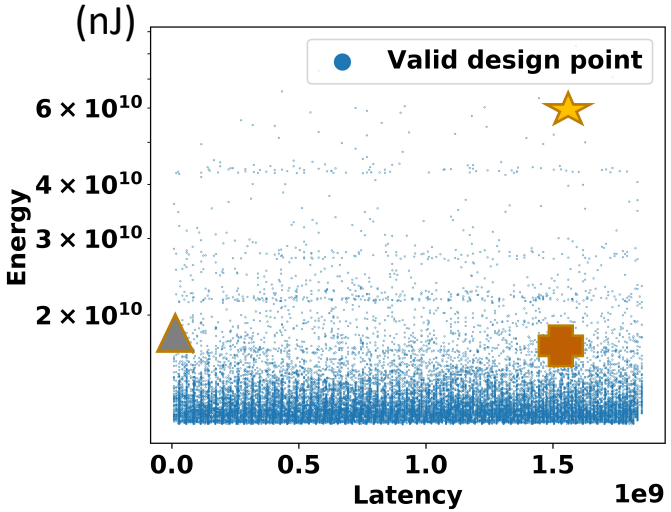
Architecture of DNN Accelerator

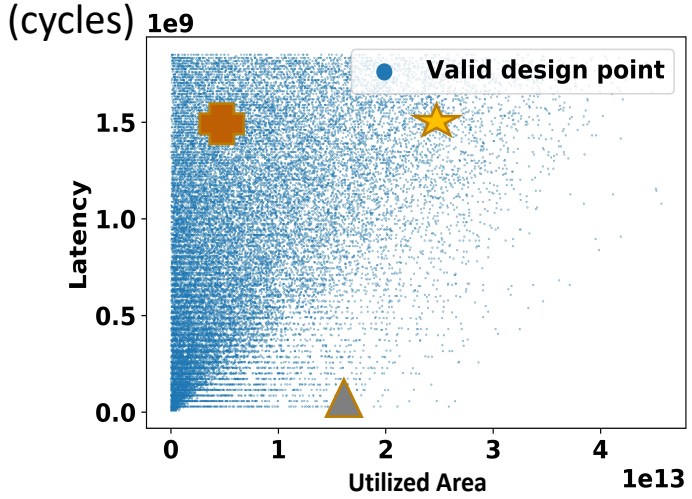


For this work, we assume the HW Resources are fixed.

Impact of Mappings

Layer: 2nd layer of VGG16 (N=1, K=64, C=64, Y=224, X=224, R=3, S=3)
HW resources: PEs: 168, SL: 512B, SG:108KB





	Min	Max
Latency (cycles)	8.18E+06	1.85E+09
Energy (nJ)	1.12E+10	8.34E+10
Power (mW)	1.52E+05	2.24E+09
Area (um^2)	1.77E+09	4.57E+13

Different mapping leads to drastically different HW performance

- Up to 4 order of magnitude difference by different mappings

Mapping Search as an Optimization Problem

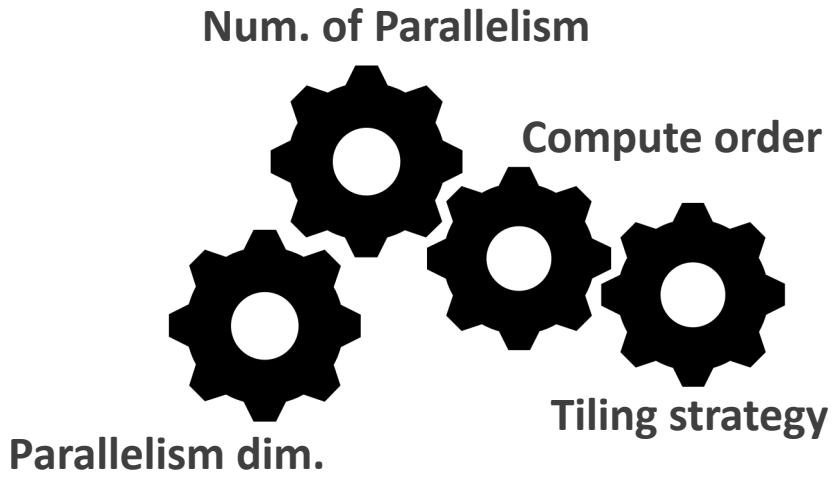
Optimization of accelerators at design/compile time

Given Platform/ accel. infrastructure

- Constrained HW resources (PEs/ buffers)

Find optimal mapping strategy

- Tune each knob of the mapping aspects

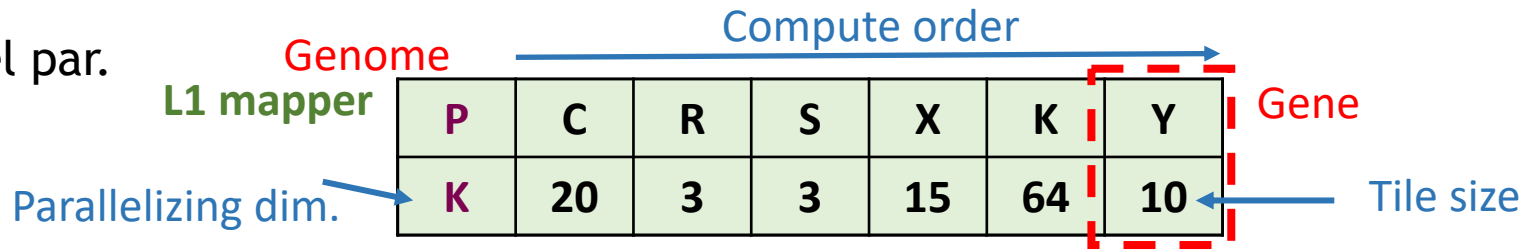


Component of mapping

Constructing the Search Space

GAMMA's genetic encoding

1-level par.



Mapping	
Compute order	✓
Tiling strategy/sizes	✓
# of Parallelism	✓
Parallelism dimension	✓

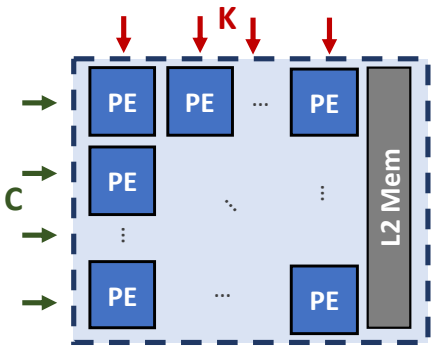
```
for c=[0, C):
    for r=[0, R):
      for s=[0, S):
        for x=[0, X):
          for k=[0, K):
            for y= [0, Y):
              output[y,x,k,c] +=
                weight[r,s,k,c] *
                  Input[y+r,x+s, k,c]
```

```
...
.....
  for k= [0, K, kt):
    for y= [0, Y, yt):
      for ct=[0, C, 20):
        for rt=[0, R, 3):
          for st=[0, S, 3):
            for xt=[0, X, 15):
              for kt=[0, K, 64):
                for yt= [0, Y, 10):
                  output[yt,xt,kt,ct] +=
                    weight[rt,st,kt,ct] *
                      Input[yt+rt,xt+st, kt,ct]
```

```
...
.....
  for k= [0, K, kt):
    for y= [0, Y, yt):
      for ct=[0, C, 20):
        for rt=[0, R, 3):
          for st=[0, S, 3):
            for xt=[0, X, 15):
              par_for kt=[0, K, 64):
                for yt= [0, Y, 10):
                  output[yt,xt,kt,ct] +=
                    weight[rt,st,kt,ct] *
                      Input[yt+rt,xt+st, kt,ct]
```

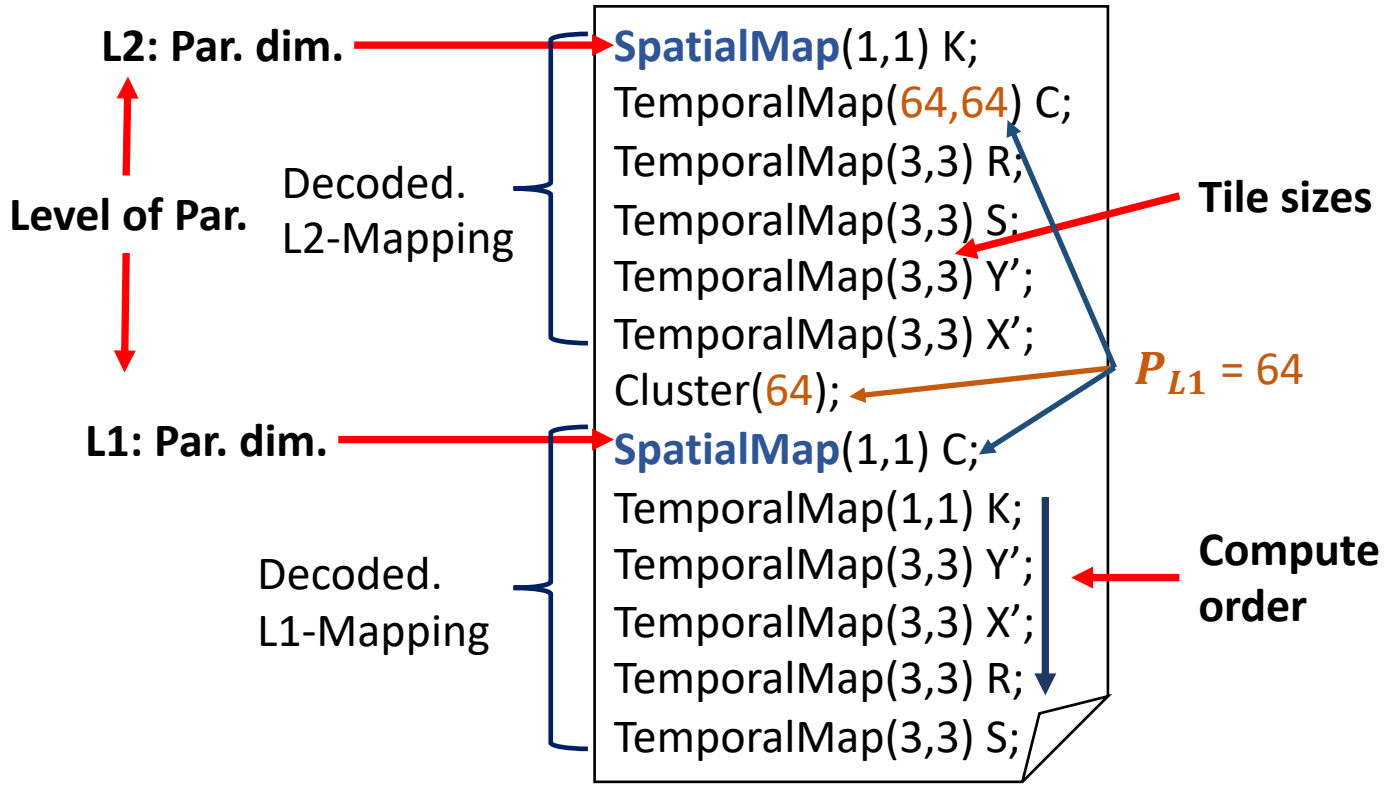
Example: NVDLA-like Mapping

P	K	C	R	S	Y	X	P	K	C	R	S	Y	X
K	1	64	3	3	3	3	C,64	1	1	3	3	3	3



NVDLA-like

- 2-level of parallelism
- Parallelize across K, C dimension



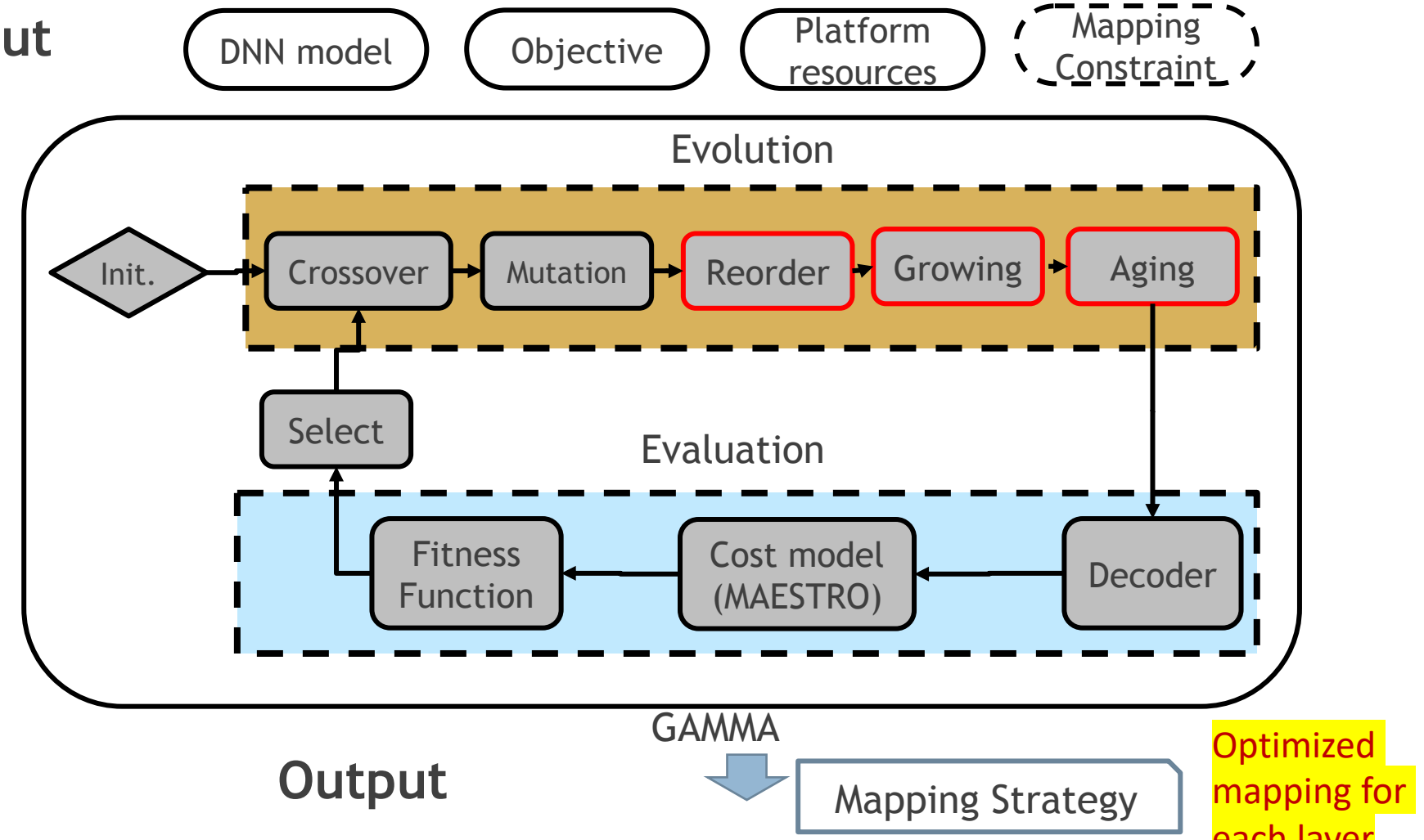
GAMMA: GA-based Optimization

Evolution

- 3 additional genetic operators

Evaluation

- Evaluate fitness
  (targeting objective: e.g., Latency)
- HW cost model (MAESTRO)

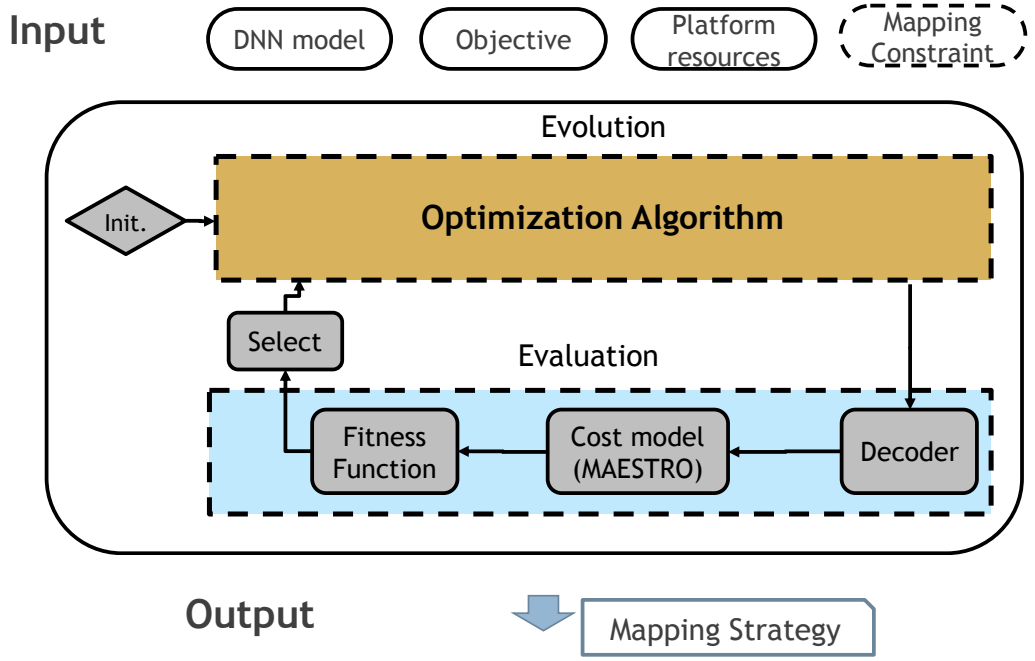


More detail: S.-C. Kao, T Krishna, "GAMMA: Automating the HW Mapping of DNN Models on Accelerators via Genetic Algorithm", ICCAD, 2020

Conventional Optimization Methods

Supported optimization methods

Alg.	Description
RS	Random Search. We randomly sample design points and keeps the best solution.
GA	Genetic Algorithm. We encode the design point into a series of genes. We evolve the genes by mutation and crossover.
DE	Differential Evolution. In DE, we mutate by the *difference vector*. When mutation, we sample two parents and extract their difference on each dimension to formulate a *difference vector*. Next, we sample another parent and mutate it by adding the *difference vector* to it.
(1 +λ)-ES	(1 +λ)-Evolution Strategy. For each parent we mutate it to reproduce λ number of mutated children. The parent and children compete with each other by their fitness values and the best one will go to the next generation.
CMA-ES	Covariance Matrix Adaptation-ES. We use covariance matrix of the elite group and the entire population to estimate the distance to the optimum. We adapt the step size based on covariances.
TBPSA	Test-based Population-Size Adaptation. We estimate the trend of the population's fitness values. When the fitness values converge, the algorithm will adapt to have a smaller population size.
PSO	Particle Swarm Optimization. We track *global best* and *parent best*, which represent the global and local information respectively. We update the parameters by the weighted sum of them.
pPort-folio	Passive Portfolio. We maximize diversity by spreading the risk broadly to multiple instances (optimizers). We launch K numbers of optimization methods, each experiencing 1/K samples, and take the best performing solution provided by one of them.



https://facebookresearch.github.io/nevergrad/

GAMMA Setup

- Setup
  - git clone https://github.com/maestro-project/gamma.git
  - conda create --name gammaEnv python=3.6
  - conda activate gammaEnv
  - pip install -r requirements.txt
  - python build.py

- Run GAMMA
  - ./run_gamma_with_hwconfig.sh

- Run other optimizations
  - ./run_others_with_hwconfig.sh

GAMMA Code-base

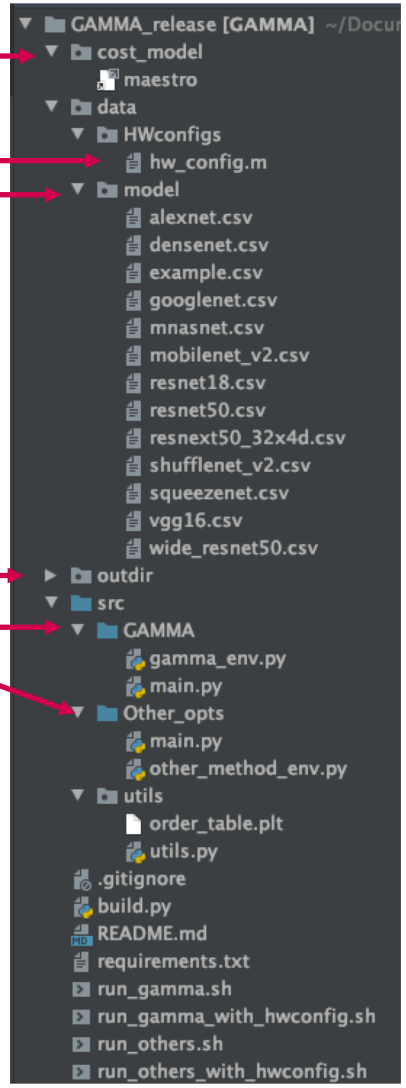
Soft-link to maestro

User-defined HW configuration

Supported model descriptions

Result directory

Main program



Code available: https://github.com/maestro-project/gamma

User Options

```
python main.py --fitness1 latency --fitness2 power --stages 1 --num_pe 168 --l1_size 512 --l2_size 108000 \ --NocBW
81920000 --slevel_min 1 --slevel_max 2 --epochs 10 \
        --model vgg16 --singlelayer 1

python main.py --fitness1 latency --fitness2 power --stages 1 --slevel_min 1 --slevel_max 2 --epochs 10 \
        --model vgg16 --singlelayer 1 --hwconfig hw_config.m
```

Objective

* fitness1/fitness2: The first/second fitness objective (latency/ power/ energy)
* stages: Number of stages, can choose from [1 or 2]
  * Choose 1, the system will only optimize on fitness1
  * Choose 2, the system will optimize on fitness1 and then fitness2
* model: The model to run
* singlelayer: The layer index of the selected model, if want to optimize only single layer. If want to optimize all layers, skip this specification.

Constraint

* num_pe: Number of PEs
* l1_size/l2_size: L1/L2 size (Bytes)
* slevel_min: The minimum number of parallelism
* slevel_max: The maximum number of parallelism, The parallelism goes from [slevel_min, slevel_max]
* hwconfig: Read in HW configuration from file

Hyper parameters

* epochs: Number of generation for the optimization
* method: The optimization methods to choose from (PSO/ Portfolio/ OnePlusOne/ CMA/ DE/ TBPSA/ pureGA/ Random/ GA)
* outdir: The output result directory

Probability of each genetic operator can be set at codebase

Add your own HW Configuration

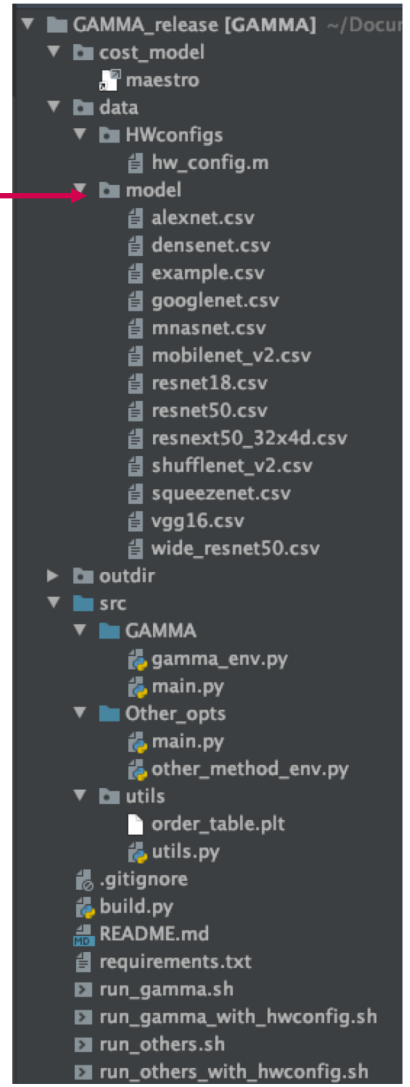
Inside ./data/HWconfigs/hw_config.m

NumPEs: *168*
L1Size: *512*
L2Size: *108000*
NoC_BW: *81920000*

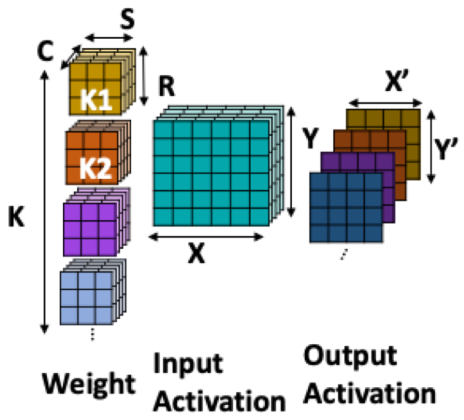
User can also set the HW configuration with hw_config, rather than from argument

Add your own DNN Model

Add new model descriptions here



CONV2D



alexnet

K	C	Y	X	R	S	T
64	3	224	224	11	11	1
192	64	27	27	5	5	1
384	192	13	13	3	3	1
256	384	13	13	3	3	1
256	256	13	13	3	3	1

[Advanced User] GAMMA-specific Tuning

In ./src/GAMMA/gamma_env.py

Tune number of population/ generation

```
def run(self, dimension, stage_idx, prev_stage_value=0, num_population=100, num_generations=100, elite_ratio=0.05,
                    parents_ratio=0.15, ratio_decay=1, num_finetune=1, best_sol_1st=None):
```

Ratio of elite we kept in the next generation. Here we kept 5% of populations.

Ratio of parents we use to do cross over.

In genetic operator, alpha is the mutation ratio.

```
def crossover_tile(self, parents, pop, alpha=0.5):
```

[Advanced User] Add your own Optimization Algorithm

In src/Other_opts/main.py

Swap to your own algorithm

Define the search space.
The range of your parameter.
(Here we define in nevergrad format.
You can use any format e.g., numpy)

Define your optimizer function here.
(Evolution block)

```
def ng_search(env,num_generations=100,num_population=100,dimension=None,choose_optimizer=None):
    parametrization = ng.p.Instrumentation(
    indv0=ng.p.Scalar(lower=0, upper=3).set_integer_casting(),
    indv1=ng.p.Scalar(lower=0, upper=720-1).set_integer_casting(),
    indv2=ng.p.Scalar(lower=1, upper=dimension[0]).set_integer_casting() if dimension[0]>1 else 1,
    indv3=ng.p.Scalar(lower=1, upper=dimension[1]).set_integer_casting() if dimension[1]>1 else 1,
    indv4=ng.p.Scalar(lower=1, upper=dimension[2]).set_integer_casting() if dimension[2]>1 else 1,
    indv5=ng.p.Scalar(lower=1, upper=dimension[3]).set_integer_casting() if dimension[3]>1 else 1,
    indv6=ng.p.Choice(np.arange(1, dimension[4], 2)) if dimension[4]>1 else 1,
    indv7=ng.p.Choice(np.arange(1, dimension[5], 2)) if dimension[5]>1 else 1 )
    optimizer = ng.optimizers.registry[choose_optimizer](parametrization=parametrization, budget=num_generations* num_population, num_workers=1)
    partial_func = partial(eval_function,env)
    recommendation = optimizer.minimize(partial_func)
    answer = recommendation.kwargs
```

Evaluate your parameters
(Evaluation block):
Modify this to take in your input parameters.

```
def eval_function(env, indv0, indv1, indv2, indv3, indv4, indv5, indv6, indv7):
    indv = [indv0, indv1, indv2, indv3, indv4, indv5, indv6, indv7]
    return get_reward(env, indv)
```

This will handle the HW performance evaluation for you.

Resources

Paper:

S.-C. Kao, T Krishna, "GAMMA: Automating the HW Mapping of DNN Models on Accelerators via Genetic Algorithm", ICCAD, 2020

Repo:

https://github.com/maestro-project/gamma